Human review packet: GGRS26CombattingGerrymanderingRCV

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GGRS26CombattingGerrymanderingRCV
Paper id: `GGRS26CombattingGerrymanderingRCV`
Source version: not recorded
Source record: `audit/paper_statement_map.json`
Rows: 2
Generated: 2026-08-18
Source URL: official source

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GGRS26CombattingGerrymanderingRCV --serve
```

Open `http://127.0.0.1:8765/` in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

Material library prerequisites

`EconCSLib.SocialChoice.Voting.Ballot`

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

```
In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each
voter instead submits a ranking over candidates (here, we assume the rankings are complete, not
partial). Consider a district with Nk seats and Vk voters; the set of winners is created iteratively, as
follows. The number of first-place votes for each candidate is counted. Any candidate with a number
of votes at least the “Droop” threshold Q = ⌊ NVk k+1 ⌋ + 1 is selected as a winner, and their surplus
votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has
enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated,
and their votes are transferred. The process continues until Nk candidates have been selected, or
the number of remaining candidates equals the number of remaining seats. STV is commonly used
in practice in multi-winner elections.
    Details can differ in how such votes are transferred, but our results in Section 3 do not depend
on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV
rule, in which each voter keeps a fractional number of votes equal to their share of the winning
candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we
assume that ties are broken in favor of candidates from party D, but randomly within each party.5
```

Lean-expanded library semantic target

```
(Candidate : Type u_1) → List Candidate
```

Exact Lean library declaration

```
abbrev Ballot (Candidate : Type*) := List Candidate
```

Recorded source-to-library screening Verdict: `matches`

Reason: The Lean target is a finite ordered candidate list, which is the source’s complete ranking submitted by each voter.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.SocialChoice.Voting.Ballot.Valid`

Source locator: cited publication, lines 394-400

Verbatim paper-source connection

```
In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each
voter instead submits a ranking over candidates (here, we assume the rankings are complete, not
partial). Consider a district with Nk seats and Vk voters; the set of winners is created iteratively, as
follows. The number of first-place votes for each candidate is counted. Any candidate with a number
of votes at least the “Droop” threshold Q = ⌊ NVk k+1 ⌋ + 1 is selected as a winner, and their surplus
votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has
enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated,
```

Lean-expanded library semantic target

```
∀ {Candidate : Type u_1} (ballot : EconCSLib.SocialChoice.Voting.Ballot Candidate), List.Nodup ballot
```

Exact Lean library declaration

```
def Valid {Candidate : Type*} (ballot : Ballot Candidate) : Prop :=
  ballot.Nodup
```

Recorded source-to-library screening **Verdict:** `matches`

Reason: The source assumes each voter submits a complete ranking over candidates. The Lean predicate records the ordinary ranking condition that no candidate is listed twice.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.SocialChoice.Voting.Ballot.nextActive`

Source locator: cited publication, lines 394-400

Verbatim paper-source connection

```
In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each
voter instead submits a ranking over candidates (here, we assume the rankings are complete, not
partial). Consider a district with Nk seats and Vk voters; the set of winners is created iteratively, as
follows. The number of first-place votes for each candidate is counted. Any candidate with a number
of votes at least the “Droop” threshold Q = ⌊ NVk k+1 ⌋ + 1 is selected as a winner, and their surplus
votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has
enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated,
```

Lean-expanded library semantic target

```
{Candidate : Type u_1} →
  [inst : DecidableEq Candidate] →
    (ballot : EconCSLib.SocialChoice.Voting.Ballot Candidate) →
      (active : Finset Candidate) →
        List.brecOn ballot fun ballot f =>
          (match (motive :=
              (ballot : EconCSLib.SocialChoice.Voting.Ballot Candidate) → List.below ballot → Option Candidate)
              ballot with
            | [] => fun x => none
            | c :: rest => fun x => if c ∈ active then some c else x.1)
            f
```

Exact Lean library declaration

```
def nextActive {Candidate : Type*} [DecidableEq Candidate]
    (ballot : Ballot Candidate) (active : Finset Candidate) : Option Candidate :=
  match ballot with
  | [] => none
  | c :: rest => if c ∈ active then some c else nextActive rest active
```

Recorded source-to-library screening **Verdict:** `matches`

Reason: STV uses each voter’s ordered preferences as candidates are selected or eliminated. The Lean function returns the first remaining active candidate in that ranking.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.SocialChoice.Voting.STVQuota`

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

```
In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each
voter instead submits a ranking over candidates (here, we assume the rankings are complete, not
partial). Consider a district with Nk seats and Vk voters; the set of winners is created iteratively, as
follows. The number of first-place votes for each candidate is counted. Any candidate with a number
of votes at least the “Droop” threshold Q = ⌊ NVk k+1 ⌋ + 1 is selected as a winner, and their surplus
votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has
enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated,
and their votes are transferred. The process continues until Nk candidates have been selected, or
the number of remaining candidates equals the number of remaining seats. STV is commonly used
in practice in multi-winner elections.
    Details can differ in how such votes are transferred, but our results in Section 3 do not depend
on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV
rule, in which each voter keeps a fractional number of votes equal to their share of the winning
candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we
assume that ties are broken in favor of candidates from party D, but randomly within each party.5
```

Lean-expanded library semantic target

```
(seats voters : ℕ) → voters / (seats + 1) + 1
```

Exact Lean library declaration

```
def STVQuota (seats voters : ℕ) : ℕ :=
  voters / (seats + 1) + 1
```

Recorded source-to-library screening **Verdict:** `matches`

Reason: The Lean target is the stated Droop threshold, voters divided by seats plus one and then plus one, used to decide whether a candidate is elected.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.SocialChoice.Voting.pavWeight`

Source locator: cited publication, lines 377-388

Verbatim paper-source connection

```
A Thiele rule is characterized by a function λ. Each voter v in district k provides a set Sv
of the Nk candidates they like most. The set of winners is determined as follows. Consider a
potential set of winners C. The amount of points that voter v contributes to C is i=1         λ(i), and
                                                                                  P|Sv ∩C|

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing)
function λ establishes that votes have diminishing returns; the ith approved candidate in the set is
worth λ(i). For example, winner-take-all approval voting is a Thiele rule with λ(i) = 1: for each
voter, the set C receives a number of points equal to the number of candidates in the set that the
voter likes, and so the Nk candidates with the most votes win. We further consider Proportional
Approval Voting (PAV), a Thiele rule with λPAV (i) = 1i ; and Thiele Squared, with λTS = i12 . PAV
is well-studied in the theoretical social choice literature and comes with optimality guarantees (in
```

Lean-expanded library semantic target

```
(approved : ℕ) → if approved = 0 then 0 else (↑approved)⁻¹
```

Exact Lean library declaration

```
noncomputable def pavWeight (approved : ℕ) : ℝ :=
  if approved = 0 then 0 else (approved : ℝ)⁻¹
```

Recorded source-to-library screening **Verdict:** `matches`

Reason: The source's PAV selector uses the PAV marginal-weight function in its displayed objective. The Lean target gives the standard 1/i PAV weight for positive indices and makes the unused zero index total.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.SocialChoice.Voting.pavHarmonicSum`

Source locator: cited publication, lines 377-388

Verbatim paper-source connection

```
A Thiele rule is characterized by a function λ. Each voter v in district k provides a set Sv
of the Nk candidates they like most. The set of winners is determined as follows. Consider a
potential set of winners C. The amount of points that voter v contributes to C is i=1         λ(i), and
                                                                                  P|Sv ∩C|

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing)
function λ establishes that votes have diminishing returns; the ith approved candidate in the set is
worth λ(i). For example, winner-take-all approval voting is a Thiele rule with λ(i) = 1: for each
voter, the set C receives a number of points equal to the number of candidates in the set that the
voter likes, and so the Nk candidates with the most votes win. We further consider Proportional
Approval Voting (PAV), a Thiele rule with λPAV (i) = 1i ; and Thiele Squared, with λTS = i12 . PAV
is well-studied in the theoretical social choice literature and comes with optimality guarantees (in
```

Lean-expanded library semantic target

```
(a : ℕ) →
  Nat.brecOn a fun x f =>
    (match (motive := (x : ℕ) → Nat.below x → ℝ) x with
      | 0 => fun x => 0
      | n.succ => fun x => x.1 + EconCSLib.SocialChoice.Voting.pavWeight (n + 1))
      f
```

Exact Lean library declaration

```
noncomputable def pavHarmonicSum : ℕ → ℝ
  | 0 => 0
  | n + 1 => pavHarmonicSum n + pavWeight (n + 1)

@[simp] theorem pavHarmonicSum_zero : pavHarmonicSum 0 = 0 := rfl
```

Recorded source-to-library screening **Verdict:** `matches`

Reason: Lemma C.1 displays the PAV objective as sums of PAV marginal weights over the two parties' selected candidates. The Lean recursion is that harmonic partial sum.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

`EconCSLib.SocialChoice.Voting.roundedSeatShare`

Source locator: cited publication, lines 485-490

Verbatim paper-source connection

```
Proposition 1 (Seat shares under STV). Suppose — for a given district with M seats and V ≥
M (M + 1) voters — that a fraction yp of voters belong to each party p ∈ {R, D}, and that there are
at least M candidates per party. Assume that each party's voters rank all same-party candidates
above all other-party candidates, and ties are broken in party D's favor.
    Let nR (yR , ST V ) be the number of winners belonging to party R using STV. Then, both
nR (yR , ST V ) and nR (yR , λPAV ) are in {⌊yR M ⌋, ⌈yR M ⌉}.
```

Lean-expanded library semantic target

```
∀ (seatCount : ℕ) (partyShare : ℝ) (seats : ℕ), seatCount = ⌊partyShare * ↑seats⌋₊ ∨ seatCount = ⌈partyShare * ↑seats⌉₊
```

Exact Lean library declaration

```
def roundedSeatShare (seatCount : ℕ) (partyShare : ℝ) (seats : ℕ) : Prop :=
  seatCount = ⌊partyShare * (seats : ℝ)⌋₊ ∨
    seatCount = ⌈partyShare * (seats : ℝ)⌉₊
```

Recorded source-to-library screening **Verdict:** `matches`

Reason: Proposition 1 states membership in the two-element floor-or-ceiling set for vote share times seats. The Lean predicate is exactly that disjunction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 2414-2433

Verbatim source input

```
Lemma C.1. Let yR ∈ (0, 1]. Consider
                                                                                    n
                                                               "           "
                                      nR (yR , λP AV ) = min arg max yR
                                                                                    X
                                                                                          λ(i)
                                                          n         n
                                                                                    i=1
                                                                             M −n
                                                                                           ##
                                                              +(1 − yR )
                                                                             X
                                                                                    λ(i)        .
                                                                             i=1

    and let ell be the unique integer such that

                                            yR (M + 1) − 1 ≤ ell < yR (M + 1)

    Then, nR (yR , λPAV ) = ell, for all yR , M .
```

Expanded PaperInterface specification

```
∀ {seats : ℕ} {partyShare : ℝ},
  0 < partyShare →
    partyShare ≤ 1 →
      ∃ seatCount,
        GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax seatCount partyShare seats ∧
          ∃ ell,
            ↑seatCount = ell ∧
              GGRS26CombattingGerrymanderingRCV.paper_pav_integer_interval ell partyShare seats ∧
                (∀ (other : ℤ),
                    GGRS26CombattingGerrymanderingRCV.paper_pav_integer_interval other partyShare seats → other = ell) ∧
                  ∀ (otherSeatCount : ℕ),
                    GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax otherSeatCount partyShare seats →
                      ↑otherSeatCount = ell
```

Lean proof endpoint (built separately)

```
GGRS26CombattingGerrymanderingRCV.paper_lemma_c1_pav_selector_eq_unique_integer_interval
```

Recorded source-to-Spec screening Verdict: `matches`

Reason: The Lean-expanded target quantifies the source domain y_R in (0,1], defines the leftmost PAV maximizer using the displayed harmonic objective, and states its equality to the unique integer in the displayed half-open interval.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 485-490

Verbatim source input

```
Proposition 1 (Seat shares under STV). Suppose — for a given district with M seats and V ≥
M (M + 1) voters — that a fraction yp of voters belong to each party p ∈ {R, D}, and that there are
at least M candidates per party. Assume that each party's voters rank all same-party candidates
above all other-party candidates, and ties are broken in party D's favor.
    Let nR (yR , ST V ) be the number of winners belonging to party R using STV. Then, both
nR (yR , ST V ) and nR (yR , λPAV ) are in {⌊yR M ⌋, ⌈yR M ⌉}.
```

Expanded PaperInterface specification

```
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
  {partyVoters otherPartyVoters allVoters : Finset Voter}
  {ballots : Voter → EconCSLib.SocialChoice.Voting.Ballot Candidate}
  {partyCandidates otherPartyCandidates initialActive : Finset Candidate} {seats voters : ℕ} {partyShare : ℝ}
  (policy :
    GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransferPolicy allVoters ballots
      ↑(EconCSLib.SocialChoice.Voting.STVQuota seats voters)),
  0 < partyShare →
    partyShare ≤ 1 →
      seats * (seats + 1) ≤ voters →
        seats ≤ partyCandidates.card →
          seats ≤ otherPartyCandidates.card →
            GGRS26CombattingGerrymanderingRCV.paper_complete_party_ranking_ballots partyVoters ballots partyCandidates
                initialActive →
              GGRS26CombattingGerrymanderingRCV.paper_complete_party_ranking_ballots otherPartyVoters ballots
                  otherPartyCandidates initialActive →
                voters = allVoters.card →
                  allVoters = partyVoters ∪ otherPartyVoters →
                    Disjoint partyVoters otherPartyVoters →
                      Disjoint partyCandidates otherPartyCandidates →
                        partyCandidates ⊆ initialActive →
                          otherPartyCandidates ⊆ initialActive →
                            initialActive ⊆ partyCandidates ∪ otherPartyCandidates →
                              partyShare * ↑voters = ↑partyVoters.card →
                                (1 - partyShare) * ↑voters = ↑otherPartyVoters.card →
                                  ∀
                                    (terminal :
                                      GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState allVoters initialActive),
                                    Relation.ReflTransGen
                                        (GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransition ballots
                                          (↑(EconCSLib.SocialChoice.Voting.STVQuota seats voters)) policy seats)
                                        (GGRS26CombattingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state
                                          allVoters initialActive)
                                        terminal →
                                      GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTerminal seats terminal →
                                        ∀ (pavSeatCount : ℕ),
                                          GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax pavSeatCount partyShare
                                              seats →
                                            GGRS26CombattingGerrymanderingRCV.paper_seat_share_rounded
                                                (GGRS26CombattingGerrymanderingRCV.ballotRoutedPartyFinalSeats
                                                  partyCandidates seats terminal)
                                                partyShare seats ∧
                                              GGRS26CombattingGerrymanderingRCV.paper_seat_share_rounded pavSeatCount
                                                partyShare seats
```

Lean proof endpoint (built separately)

```
GGRS26CombattingGerrymanderingRCV.paper_proposition1_all_ballot_routed_surplus_transfer_outcomes_and_selected_pav
```

Recorded source-to-Spec screening Verdict: `matches`

Reason: The Lean-expanded target states the source's two-party solid-coalition STV/PAV rounded-seat conclusion for every reachable ballot-routed surplus-preserving terminal outcome. The source's D-favoring tie convention is a subset of this tie-robust result, so the formal target is stronger rather than omitting a needed source instance.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation